

Key Rotation

Overview

OA4MP as of version 6.3 supports key rotation, *i.e.*, the periodic invalidation of current keys and creation of new ones to replace them. So the server keys as well as virtual issuer keys that may be rotated as well. For this to work, a **key store** which is just backed by persistent storage (*e.g.* a table in the database) is needed.

Many clients cache keys from the server, so the basic policy is to set a date for the rotation to occur and on that date, issue new valid keys, setting the expiration date for the current keys. The new keys are then made active immediately and the server switches to using them. The old keys remain available (so outstanding tokens may be refreshed/exchanged). The expiration is set to be the sum of the (expected) client's cache and the service's access token lifetime. This should give clients the opportunity to refresh their caches with the new keys and have enough time on their outstanding access tokens to refresh/exchange them.

Note that the original system whereby the keys are specified in the server's XML configuration is still supported and works., All changes are backwards compatible so if you are upgrading you do not need to enable key rotation until you are good and ready. It is also easy to back out of by simply disabling it.

Original system

This is documented [here](#). In short, a set of JSON Web Keys (JWKs) are generated and specified in the server XML configuration file. A default id is given for all signing (so that is used if no other key is specified). The main issue is that rotating the keys is awkward. That could be done by adding keys, then changing the default (requiring a server restart) and doing all ageing manually.

Each VI (virtual issuer) could have its own set of keys, and these were specified in the VI configuration, along with a default key id. Again, the only way to rotate was manually, although this did not require a server restart.

Note: The new system is *optional* meaning that you are not required to use it so if you are fine with the old system, you can ignore the new one.

The new key rotation system.

Summary.

Major features are

1. A dedicated key store that holds all current keys in the system. This centralizes key management All keys, including those for the server and VIs go here, and changes do not require a server restart.

2. All active keys are automatically displayed in the well-known page. This is updated in real time to reflect changes to the system.
3. Support for signing keys in requests. Now a client may request a specific key be used to sign a request. This must, of course, be known to the system – only keys in the well-know page are allowed.
4. CLI support for rotating keys. This makes rotating a key manually easy.
5. Serve policies for automatic rotation.

Installing the key store

You need to update the persistent storage system. In the case of a database, that means adding a table with the correct permissions. These are in the `oa4mp/server-admin/src/main/resources` directory in [GitHub](#). The file you want is named `oa4mp-X.sql`, where X is the database, derby, mysql etc. The table definition **key_records** is there. Be sure again that the table is given the correct permissions for whatever database user you use.

Once the store has been updated, you need to add the `<keyStore/>` tag to the list of components in the server's XML configuration file.

Migration from a previous version of OA4MP

If your current install uses the old system, it is best to migrate the entries to the new system. This can be done easily with the command line interface (CLI). If you have keys in the server configuration, you can just pull them into the store. Note that this needs to be done once. If there are no server keys in the database, the system will default to use what is in the configuration file, so you don't need to do this initially. If you are setting up a server and have no keys, see the section on generating keys to set up your server before its first use.

Generating keys.

If you have a server configuration with keys already, you can just migrate those into the store. This means using the CLI and once migrated, the keys in the store will override the keys in the configuration. If you wish, you may remove the keys in the configuration once this is done or keep them as a backup. Also note that the newer features (such as setting an expiration date for the keys) won't be reflected in the configuration file – it is ignored once migrated.

Adding keys to a VI

How a client requests a specific key

If a key is in the well-known page, (this works for virtual issuers as well), then the client may add the parameter `oa4mp:/jwt/signing/keyID` with the value of the **kid** (JSON web key id), e.g.

`https://oa4mp.bgsu.edu/authorize?oa4mp:/jwt/signing/keyID=AB285786E8756&...`

would start a standard authorization flow to a server and requests that the returned tokens be signed with the key whose ID is AB285786E8756. Note that this is *all* subsequent keys. You may pass this parameter in any call that returns a token and switch to signing with that key instead.

Why do this? Some services are only able to process certain types of keys (RSA, Elliptic, etc.). A virtual issuer may be configured with various types of keys for various services, so that the service that consumes the token can have a key it understands.

If an kid is used that the server does not recognize, this simply means that it is ignored and the system uses whatever the default is.

How key rotation works

At its simplest, keys should be changed according to a definite schedule, such as every month or 6 months. Clients *should* get the current keys from the well-known page. Each key has a unique id, the kid, a date of issue, an expiration date and a “not valid before” date. A key without an expiration date has an effective infinite lifetime.

Key rotation may be done manually or automatically.

Server key rotation

You may rotate the server keys using the CLI or via a timer. The plus with a timer is it happens automatically. To manually rotate the server (or any other keys) requires using the CLI.

Server key rotation configuration

The XML configuration file allows for a section on key rotation.

Virtual issuer key rotation

Each virtual issuer may have its own set of keys. These may be rotated automatically or manually too. In particular, the requirements for key rotation use the server’s unless they are overridden for the virtual issuer.